Assignment 02

Published | Assign To | Edit | ⋮

Use of Generative AI Is Not Permitted

Learning Objectives

The goal of this assignment is to continue building a cloud-native web application and manage the source code in GitHub repository.

GitHub Subscription

- All students will need to subscribe to the GitHub **Team** (https://github.com/pricing) plan.

Create & Set Up GitHub Repository

1. **Create** (https://help.github.com/articles/create-a-repo/) a new private GitHub repository for web applications in the GitHub organization you created.
2. The GitHub repository name must be `webapp`.
3. Grant TAs access to your GitHub repository.
4. Update `README.md` in your repository. Your readme file must contain the following:
    1. Prerequisites for building and deploying your application locally.
    2. Build and deploy instructions for the web application.
5. Fork the GitHub repository in your namespace. You will do all development work on your fork.
6. All web application code should now be in this repository.
7. Add appropriate `.gitignore` to your repository. A collection of useful `.gitignore` templates can be found **here** (https://github.com/github/gitignore).

Cloud-Native Web Application Requirements

RESTful API Endpoints To Be Implemented

About the field data types in Swagger docs

1. If a field has `readOnly` set to `true`, the value in this field is populated by your application. Example fields are timestamp and id. `readOnly` properties are included in responses but not in requests.
2. If a field has `writeOnly` set to `true`, the value in this field is provided by the API caller in `POST` or `PUT` request, but these fields are not part of the `GET` request. An example is a `password` field. `writeOnly` properties may be sent in requests but not in responses.
3. The keyword `multipleOf` is used to specify that a number must be a multiple of another number.

4. The `minimum` and `maximum` keywords are used to specify the range of possible values.

API Documentation

Swaggerhub: **https://app.swaggerhub.com/apis-docs/csye6225-webapp/cloud-native-webapp/fall2025-a2 (https://app.swaggerhub.com/apis-docs/csye6225-webapp/cloud-native-webapp/fall2025-a2)**

Requirements

1. All API request/response payloads should be in JSON.
2. No UI should be implemented for the application.
3. As a user, I expect all API calls to return with a proper **HTTP status code (https://en.wikipedia.org/wiki/List_of_HTTP_status_codes)** .
4. ~~As a user, I expect the application's code quality to be maintained to the highest standards using unit and/or integration tests.~~
5. Your web application must only support **Token-Based authentication and not Session Authentication (https://security.stackexchange.com/questions/81756/session-authentication-vs-token-authentication)** .
6. As a user, I must provide a **basic (https://en.wikipedia.org/wiki/Basic_access_authentication#Client_side)** **authentication (https://developer.mozilla.org/en-US/docs/Web/HTTP/Authentication)** token when making an API call to the `authenticated` endpoint.
7. Create a new user
    1. As a user, I want to create an account by providing the following information.
        1. Email Address
        2. Password
        3. First Name
        4. Last Name
    2. `account_created` field for the user should be set to the current time when user creation is successful.
    3. Users should not be able to set values for `account_created` and `account_updated`. Any value provided for these fields must be ignored.
    4. `Password` should never be returned in the response payload.
    5. As a user, I expect to use my *email address* as my *username*.
    6. The application must return `400 Bad Request` HTTP response code when a user account with the email address already exists.
    7. As a user, I expect my password to be stored securely using the **BCrypt password hashing scheme (https://docs.spring.io/spring-security/site/docs/current/apidocs/org/springframework/security/crypto/bcrypt/BCrypt.html)** with **salt (https://en.wikipedia.org/wiki/Salt_(cryptography))** .
8. Update user information

1. As a user, I want to update my account information. I should only be allowed to update the following fields.
    1. First Name
    2. Last Name
    3. Password
2. Attempt to update any other field should return `400 Bad Request` HTTP response code.
3. `account_updated` field for the user should be updated when the user update is successful.
4. A user can only update their own account information.

9. Get user information
    1. As a user, I want to get my account information. The response payload should return all fields for the user except for `password`.
10. Add Product
    1. Any user can add a product.
    2. Product quantity cannot be less than 0.
11. Update Product
    1. Only the user who added the product can update the product.
    2. Users can use either the PATCH or PUT API for updates.
12. Delete Product
    1. Only the user who added the product can delete the product.

Assignment Submission Instructions

Development Workflow

1. **Branch Management**
    - Complete all work on a feature branch in your forked repository
    - Merge your feature branch to `main` only when development is complete
    - Ensure your feature branch and `main` branch are synchronized before submission

Submission Requirements

Step 1: Create Submission Folder

Create a folder using this naming convention:

```
firstname_lastname_neuid_a_##
```

Replace `##` with the assignment number (e.g., `01`, `02`, etc.)

Step 2: Clone Repositories

1. Navigate to your submission folder
2. Clone **all** required GitHub repositories using:

bash

```
git clone [repository-url]
```

3. **Important:** Use `git clone` to preserve your complete commit history and branch information

Step 3: Create and Verify ZIP File

1. Create a ZIP file of your submission folder:

```
firstname_lastname_neuid_a_##.zip
```

2. **Verification Step:** Extract your ZIP file in a different location and confirm:
   - All repositories are present
   - Commit history is intact
   - All required branches exist

Step 4: Submit

Upload your ZIP file to the assignment submission portal.

Important Notes

- **Multiple Submissions Allowed:** You may resubmit if needed. Only your latest submission will be graded.
- **Quality Check:** Always verify your ZIP file contents before final submission to avoid issues.
- **Complete Repositories Required:** Submit code from all repositories mentioned in the assignment requirements.

Troubleshooting

If you encounter issues with your ZIP file, feel free to resubmit after verification. Take time to ensure all requirements are met before your final submission.

Cloud-Native Web Application Grading Guidelines

System Stability & Performance

- **No Server Errors**: Application must not generate 500 internal server errors during testing
- **Application Uptime**: No restart of the application server required between API calls
- **Error Handling**: Application should handle invalid requests gracefully with appropriate HTTP status codes

Demo Requirements

- **Platform**: Students must demonstrate the web application from their own laptops
- **API Testing**: APIs must be tested using REST clients (Postman, Restlet, Insomnia, etc.)
  - Browser-based testing is **not acceptable**
- **UI Restriction**: Application must **not** have any user interface components
  - Verify complete absence of HTML pages, forms, or visual components

Security Implementation

Authentication & Authorization

- **Authentication Method**: Must use Basic Authentication with a token-based approach
  - Session-based authentication is **not allowed**
  - JWT tokens are **not allowed**
- **Password Security**: All passwords must be encrypted using BCrypt with salt
  - Verify in the database that passwords are properly hashed
  - Check that salt is unique per password

Data Protection

- **Response Payload Security**: Password fields must **never** be included in API responses
  - Test all endpoints that return user data
- **Username Validation**: Only valid email addresses can be used as usernames
  - Test rejection of non-email username formats

Data Integrity & Business Logic

Account Management

- **Duplicate Prevention**: Application must prevent the creation of multiple accounts with the same email address
  - Test duplicate email registration scenarios
  - Verify appropriate error responses for duplicate attempts
- **Audit Fields Protection**: System-managed fields must be read-only
  - `account_created` field: Set automatically by application, never by user input
  - `account_updated` field: Set automatically by application, never by user input
  - Test attempts to manually set these values through API calls

Code Quality & Standards

Configuration Management

- **No Hard-Coding**: Variable values must be configurable (environment variables, config files)
- **Default Values**: Acceptable when overrideable during runtime or demos
- **Demo Flexibility**: Values must be modifiable when requested by the instructor/TA

Code Style & Documentation

- Follow industry-standard style guides for your programming language
- Include meaningful comments for complex logic
- Maintain consistent formatting and naming conventions
- Provide clear README files with setup and usage instructions

Application Stability & Error Handling

Runtime Stability

- **No Crashes**: Application must handle API calls without crashing
- **No Restarts**: Application/VM should not require restarts during demos (unless specifically required for feature demonstration)
- **Graceful Degradation**: Handle edge cases and invalid inputs appropriately

Error Management

- **No 500 Errors**: All server errors must be properly handled
- **Meaningful Error Messages**: Provide clear, actionable error responses
- **Input Validation**: Validate and sanitize all user inputs
- **Resource Management**: Properly handle database connections, file operations, and external API calls

Edge Case Handling

- Test boundary conditions (empty inputs, maximum values, null cases)
- Handle network timeouts and connection failures
- Implement appropriate fallback mechanisms
- Validate data integrity throughout the application lifecycle

Testing Checklist

Pre-Demo Verification

- Application starts without errors
- Database connectivity confirmed
- REST client configured and ready

Core Functionality Tests

- Account creation with a valid email
- Account creation rejection with an invalid email format
- Duplicate account creation prevention
- Password encryption verification in the database
- Authentication flow testing
- Response payload validation (no password fields)

- Audit field protection testing

Error Handling Tests

- Invalid request handling
- Malformed data rejection
- Appropriate HTTP status codes returned
- Server stability under error conditions

Evaluation Criteria

- **Pass**: All requirements met, no critical issues identified
- **Conditional Pass**: Minor issues that don't compromise core functionality or security
- **Fail**: Any security vulnerabilities, system crashes, or fundamental requirement violations

Common Issues to Watch For

- Exposed passwords in API responses
- Session-based authentication implementation
- UI components present in the application
- Improper error handling leading to crashes
- Missing input validation
- Audit trail manipulation vulnerabilities

Grading Guidelines Git Workflow & Repository Management

Repository Setup & Access

- **Fork Verification**: Confirm the student has created a proper fork of the organization's repository
- **Collaborator Access**: Verify TAs have been added as collaborators with appropriate permissions
- **Repository Location**: Ensure the dev environment is **not** set up in `Downloads` folder or other inappropriate locations

Git Protocol & Configuration

- **SSH Protocol**: Verify the repository was cloned using `git/ssh` protocol (not `https`)
  - **Verification method**: Run `git remote -v` in the cloned repository
  - **Expected output**: URLs should begin with `git@github.com:` not `https://`
- **Remote Configuration**: Ensure the upstream remote is properly configured for the original organization repository

Branch Management & Workflow

- **Main Branch Sync**: Verify `main` branch contains all assignment code and is properly synchronized
  - **Verification method**: Have students create pull requests between `main` and assignment branches
  - **Expected result**: No uncommitted changes should exist
- **Commit History**: Confirm no direct commits were made to the organization repository (all work done on a fork)
- **Branch Strategy**: Verify proper use of feature branches if required by assignment specifications

Documentation

- **README.md Requirements**: Must include comprehensive setup instructions covering:
  - Prerequisites (programming language versions, system requirements)
  - Framework and library dependencies
  - Step-by-step build instructions
  - Testing procedures and commands
  - Deployment instructions
  - Any environment-specific configuration needed

Repository Content & Hygiene

File Exclusion Standards

- **IDE Files**: No IDE-specific configuration files (`.idea/` , `.vscode/` , `.eclipse/` , etc.)
- **Build Artifacts**: No compiled files or build outputs
  - Java: No `.class` , `.jar` , `.war` files
  - Node.js: No `node_modules/` directory
  - General: No `build/` , `dist/` , `target/` directories
- **System Files**: No OS-specific files (`.DS_Store` , `Thumbs.db` , etc.)

Dependency Management

- **External Dependencies**: Third-party libraries from package managers (Maven, npm, pip) must **not** be committed
- **Gitignore Configuration**: Verify `.gitignore` file is properly configured and comprehensive
  - Must include language-specific patterns
  - Must exclude build artifacts and dependencies
  - Should include common IDE and OS files

Verification Checklist

Pre-Grading Setup

- Clone the student's fork locally
- Verify repository access and permissions

- Check the remote configuration with `git remote -v`

Content Review

- Review commit history for direct commits to the organization repo
- Scan the repository for excluded file types
- Test README instructions in a clean environment
- Verify `.gitignore` effectiveness

Workflow Validation

- Check branch synchronization
- Validate fork relationship
- Confirm proper Git protocol usage

Common Issues & Deductions

Major Issues

- Repository cloned via HTTPS instead of SSH
- Direct commits to the organization's repository
- Missing or inadequate README.md
- Dependencies committed to the repository

Minor Issues

- IDE files present but minimal
- Incomplete `.gitignore` configuration
- Minor documentation gaps
- Incorrect repository location

Zero Tolerance Issues (Automatic significant deduction)

- No fork created
- TAs not added as collaborators
- Repository not accessible
- Complete absence of required documentation

Demo Expectations

- The application should run smoothly without intervention
- All features must be demonstrable as specified
- Configuration changes should be possible when requested
- Error scenarios should be handled gracefully

Use of Generative AI Is Not Permitted

Points	10
Submitting	a file upload
File Types	zip
Allowed Attempts	3

Due	For	Available from	Until
Sep 25 at 3pm	Everyone	Sep 18 at 6pm	Sep 26 at 3pm

+ Rubric